DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

DELHI TECHNOLOGICAL UNIVERSITY

(Formerly Delhi College of Engineering)

Shahbad Daulatpur, Main Bawana Road, Delhi – 110042

EMBEDDED SYSTEMS PROJECT REPORT

Wi-Fi CONTROLLED RC CAR

Using STM32 Blue Pill, NodeMCU and L298N Motor Drivers

Academic Year 2025-26

Submitted By

Sujal Kumar Singh

CERTIFICATE

This is to certify that the project report entitled **"Wi-Fi Controlled RC Car using STM32 Blue Pill, NodeMCU and L298N Motor Drivers"** is a Bonafide record of the embedded systems project work carried out by the following student of B.Tech. (Electronics and Communication Engineering) at Delhi Technological University during the academic year 2025-26:

Name	Roll Number
Sujal Kumar Singh	23/EC/207

The work presented in this report has been carried out under our supervision and to the best of our knowledge has not been submitted elsewhere for the award of any degree or diploma. The candidates have completed the project satisfactorily as per the curriculum requirements.

_____

Project Mentor / Supervisor
Department of ECE
Delhi Technological University

_____

Head of Department
Department of ECE
Delhi Technological University

Date: _____
Place: Delhi

DECLARATION

We, the undersigned students of B.Tech. Electronics and Communication Engineering at Delhi Technological University, hereby declare that the project report entitled "Wireless Wi-Fi Controlled RC Car using STM32 Blue Pill, NodeMCU and L298N Motor Drivers" submitted to the Department of Electronics and Communication Engineering, is an original record of our own work carried out under the guidance of our project mentor.

We further declare that the matter embodied in this project report has not been submitted, either in part or in full, to any other university or institution for the award of any degree, diploma or certificate. All external sources of information used in this work have been duly acknowledged in the references section. We bear full responsibility for the originality of the content and design decisions presented herein.

Name	Signature
Sujal Kumar Singh (23/EC/207)	_____

Date: _____
Place: Delhi

ACKNOWLEDGMENT

We would like to express our sincere gratitude to the Department of Electronics and Communication Engineering, Delhi Technological University, for providing us with the opportunity to undertake this embedded systems project on the design and development of a Wi-Fi controlled RC Car. The departmental laboratory facilities and the structured curriculum exposure to microcontrollers, communication interfaces, and power electronics formed the foundation upon which this project was built.

We extend our heartfelt thanks to our faculty mentor and lab instructors for their invaluable guidance, technical insight, and constant encouragement throughout the duration of this project. Their expertise in embedded systems, microcontroller architecture, and circuit design was instrumental in helping us navigate critical design decisions and overcome several non-trivial technical challenges — including the recovery from a damaged GPIO pin and the diagnosis of UART noise from the NodeMCU boot sequence.

We also gratefully acknowledge the senior students and peers who patiently answered our questions during long debugging sessions, and the maintenance staff of the electronics laboratory for ensuring continuous availability of bench instruments. The Head of Department deserves special mention for fostering an environment that encourages independent project work and hands-on experimentation.

Finally, we thank our families for their unwavering moral support during late-night iterations and exam-period balancing, and our friends for their genuine interest in our progress that kept us motivated through every milestone. This project would not have reached completion without the collective contribution of all the people mentioned above.

Sujal Kumar Singh (23/EC/207)

ABSTRACT

This project presents the design, implementation, and verification of a fully wireless four-wheel-drive Remote-Controlled (RC) car built around the STM32F103C8T6 (Blue Pill) microcontroller and the ESP8266-based NodeMCU module. The vehicle is controlled from a smartphone over Wi-Fi through a responsive, web-based touch interface that requires no native application — any modern browser is sufficient.

The architecture follows a clean master-slave division of responsibility: the NodeMCU operates as a dedicated communication front-end hosting a Wi-Fi Soft Access Point and an embedded HTTP server, while the STM32 Blue Pill performs all real-time motor control, PWM generation, and direction logic on bare metal without any HAL abstraction. Two L298N dual H-bridge driver modules — one per side — provide independent torque control to four DC geared motors arranged in a tank-drive configuration. Differential steering is used in place of a steering mechanism, enabling sharp on-the-spot pivot turns.

The user interface implements hold-to-move semantics where buttons must be held to keep the vehicle moving. An 80-millisecond heartbeat keeps the motors active while a button is pressed, and an aggressive eight-fold stop burst on release guarantees reliable cessation of motion even under intermittent Wi-Fi packet loss. The complete control loop — from button press on the phone to motor response — exhibits end-to-end latency under 100 milliseconds in typical conditions.

The system was developed iteratively over several stages of prototyping and debugging. Major challenges encountered and resolved during development included a damaged GPIO pin (PA6) which required relocating PWM generation to PA5 with a different timer, phantom motor activation caused by ESP8266 boot-time UART noise, and Wi-Fi latency issues mitigated by disabling the ESP8266 sleep mode and shortening URL paths. The final implementation operates reliably and demonstrates a complete, end-to-end embedded systems product.

TABLE OF CONTENTS

1. INTRODUCTION

Remote Controlled (RC) vehicles have evolved significantly over the past decade, transitioning from radio-frequency based control systems to Internet-of-Things (IoT) enabled smart platforms. With the proliferation of inexpensive Wi-Fi modules and powerful 32-bit ARM-based microcontrollers, it has become feasible for student teams to build sophisticated wireless control systems using off-the-shelf components for a fraction of the cost that comparable commercial products demand.

This project demonstrates the design and implementation of a small-scale four-wheeled RC car controlled wirelessly via a smartphone over Wi-Fi. The system uses two microcontrollers in a master-slave configuration: an ESP8266-based NodeMCU module that hosts the Wi-Fi access point and the web interface, and an STM32F103C8T6 (Blue Pill) microcontroller that handles all real-time motor control, PWM generation, and direction logic.

Unlike traditional radio-controlled cars that require a dedicated transmitter, this system leverages the user's existing smartphone as the remote. A built-in web server on the NodeMCU serves an HTML interface accessible through any standard browser, making the system platform-agnostic and easy to use. The smartphone connects to the NodeMCU as if it were a normal Wi-Fi hotspot — no native application installation is required.

The project demonstrates practical implementation of several core embedded systems concepts including bare-metal C programming on ARM Cortex-M3, direct GPIO register manipulation, timer-based PWM signal generation, UART serial communication, voltage regulation using switching converters, Wi-Fi networking with the ESP8266 SDK, and embedded HTTP serving — all integrated into a cohesive, working product. This integrated approach mirrors the kind of multidisciplinary work expected in industry-grade IoT product development.

2. OBJECTIVES

The primary objectives of this project are:

- To design and assemble a four-wheel-drive RC car capable of bidirectional motion and on-the-spot pivot turning.
- To implement bare-metal firmware on the STM32 Blue Pill for real-time motor control using GPIO and timer peripherals without relying on hardware abstraction layers.
- To configure two L298N motor driver modules for independent left-side and right-side motor control with PWM-based variable speed regulation.
- To program the NodeMCU as a standalone Wi-Fi Soft Access Point hosting a mobile-responsive web control interface.
- To establish reliable UART-based communication between the NodeMCU and STM32 for command transmission with strict input validation.
- To implement a hold-to-move user interaction model with safety features such as automatic stopping on button release, page hide, or connection loss.
- To optimise network responsiveness through heartbeat-based command repetition, burst stop transmissions, and Wi-Fi sleep-mode tuning.
- To document the complete development process, including hardware design choices, software implementation details, debugging methodology, and testing outcomes.

3. LITERATURE AND BACKGROUND

3.1 STM32 Microcontroller Family

The STM32F103C8T6 (commonly referred to as the Blue Pill) is a 32-bit ARM Cortex-M3 based microcontroller manufactured by STMicroelectronics. It operates at up to 72 MHz, contains 64 KB of flash memory and 20 KB of SRAM, and offers a rich set of peripherals including multiple GPIO ports, four general-purpose timers, three USARTs, two I2C interfaces, two SPI interfaces, a USB controller, and a 12-bit ADC. Its low cost and broad feature set make it a popular choice for embedded systems education and prototyping.

3.2 ESP8266 and NodeMCU

The ESP8266 is a low-cost Wi-Fi microcontroller produced by Espressif Systems that integrates a 32-bit Tensilica L106 processor with a complete IEEE 802.11 b/g/n Wi-Fi subsystem. The NodeMCU is a popular development board built around the ESP-12E module that integrates the ESP8266, a CH340 USB-to-serial bridge, a voltage regulator, and breakout headers in a single board. The NodeMCU can be programmed using the Arduino IDE and supports operation both as a Wi-Fi client and as an access point.

3.3 L298N H-Bridge Driver

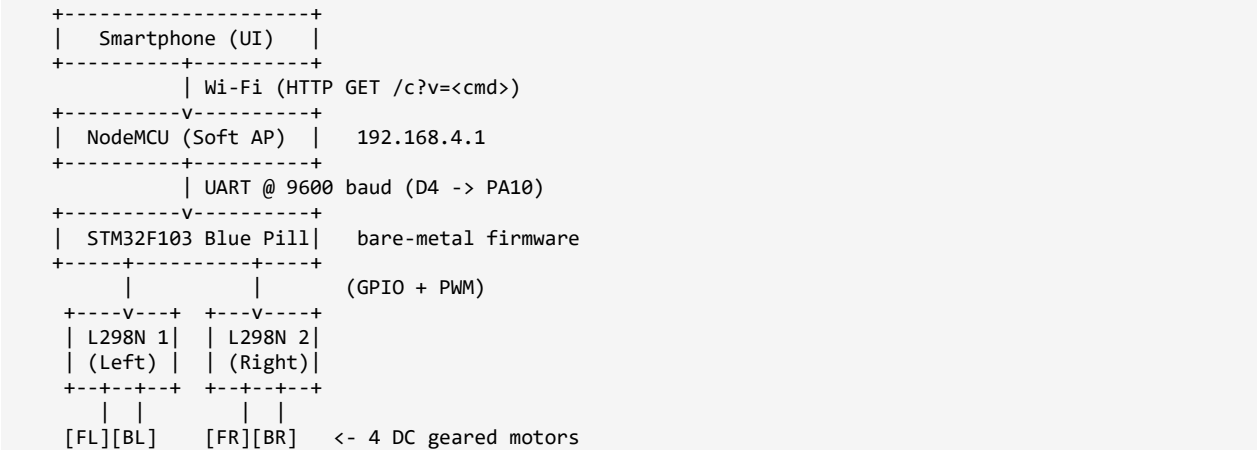
The L298N is a dual H-bridge motor driver IC capable of driving two DC motors independently with currents up to 2 A per channel and supply voltages up to 46 V. Each channel provides four output pins controlled by direction inputs (IN1, IN2 for channel A; IN3, IN4 for channel B) and a PWM-capable enable input (ENA, ENB). Most breakout boards include an on-board 5 V regulator with a jumper-selectable bypass, and removable jumpers that allow the enable lines to be either tied permanently HIGH or controlled externally with a PWM signal.

3.4 Differential Drive Steering

Since the prototype does not include a steering mechanism, motion is achieved through differential drive — a kinematic configuration where direction changes are produced by running the left-side and right-side motors at different speeds, or by completely halting one side while the other continues. This approach is widely used in tracked vehicles, robot vacuum cleaners, and small wheeled robots. Pivot turns (one side stationary, other at full speed) provide the tightest turning radius and were chosen as the steering mode for this project.

4. SYSTEM ARCHITECTURE

The system follows a layered architecture comprising three principal subsystems: the user interface layer (smartphone web browser), the communication layer (NodeMCU Wi-Fi access point and UART link), and the actuation layer (STM32 microcontroller and motor drivers). A high-level overview of the command flow is shown below.



The smartphone hosts a web browser that connects to the NodeMCU's Wi-Fi access point named "RCMotor". When the user presses a control button, the browser issues an HTTP GET request that the NodeMCU's onboard web server receives and parses. The corresponding single-character command (F, B, L, R, S, C, 1, 2, or 3) is then transmitted via the secondary UART (Serial1, GPIO2/D4) at 9600 baud to the STM32's USART1 receive pin (PA10). The STM32 firmware validates the received character against the supported command set, ignores anything else, and drives the appropriate motor direction pins and PWM signals. The L298N modules amplify these low-current signals into the high-current motor drive currents required to spin the four DC motors.

5. HARDWARE COMPONENTS

The complete list of hardware components used in this project is presented in the table below. Photographs and individual descriptions of each component follow in Section 5.1.

Component	Specification	Quantity
STM32F103C8T6 Blue Pill	ARM Cortex-M3, 72 MHz, 64 KB flash	1
NodeMCU V3	ESP8266 ESP-12E, 80 MHz, 4 MB flash	1
L298N Motor Driver	Dual H-bridge, 2 A per channel, 5-35 V	2
DC Geared Motors	BO type, 100 RPM, 3-12 V	4
Li-ion Battery Cells	18650 type, 3.7 V / 2200 mAh each (×3)	3
L7805 Voltage Regulator	Fixed 5 V output, TO-220 package	1
ST-Link V2	SWD programmer for STM32	1
Jumper Wires	M-M, M-F, F-F assorted	30+

Breadboard	400-point dual power rail	1
Toggle Switch (SPDT)	Main power on/off switch, 2 A	1
Chassis	Acrylic 4-wheel drive base with motors	1
USB Data Cables	Micro-USB to USB-A (data-capable)	2

5.1 Component Photographs and Descriptions

Each major component used in the build is shown below with a brief functional description. Figures are sized for clarity at print resolution.

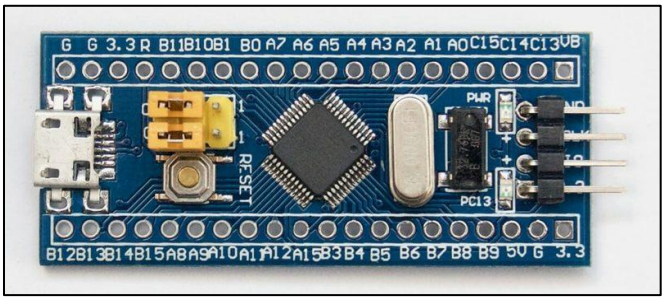


Figure 5.1 — STM32F103C8T6 Blue Pill Microcontroller

The STM32 Blue Pill serves as the master controller of the system. It is responsible for generating PWM signals for motor speed control, setting direction pins on both motor drivers, and receiving single-character commands from the NodeMCU over UART. All 13 GPIO connections to the motor drivers and one UART receive pin (PA10) interface with this board.

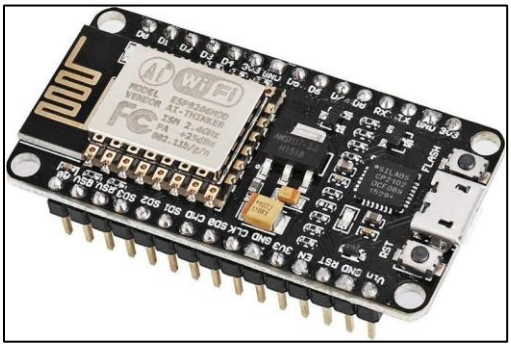


Figure 5.2 — NodeMCU V3 (ESP8266) Wi-Fi Module

The NodeMCU hosts a Wi-Fi Soft Access Point named "RCMotor" and serves the HTML-based control interface to the user's smartphone. It runs an embedded HTTP server and forwards every received command to the STM32 over its secondary UART (Serial1 on D4/GPIO2).

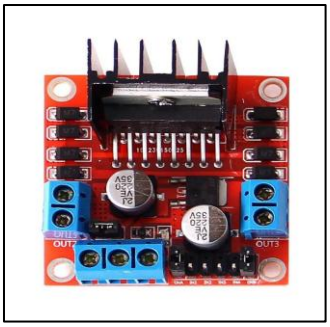


Figure 5.3 — L298N Dual H-Bridge Motor Driver

Two L298N modules are used — one to drive the two left-side motors and another for the two right-side motors. Each module accepts logic-level direction inputs (IN1-IN4) and PWM enable signals (ENA, ENB) from the STM32, and drives the high-current motor outputs from the 12.6 V battery rail. The ENA and ENB jumpers are removed so that the STM32 can control speed via PWM.

Figure 5.4 — DC Geared TT Motor with Wheel Hub

Four BO-type DC geared motors (commonly called "TT motors") provide the vehicle's drive. Each motor is rated for 3-12 V operation at approximately 100 RPM under load. The plastic gearbox provides the necessary torque multiplication for moving the loaded chassis on indoor surfaces.

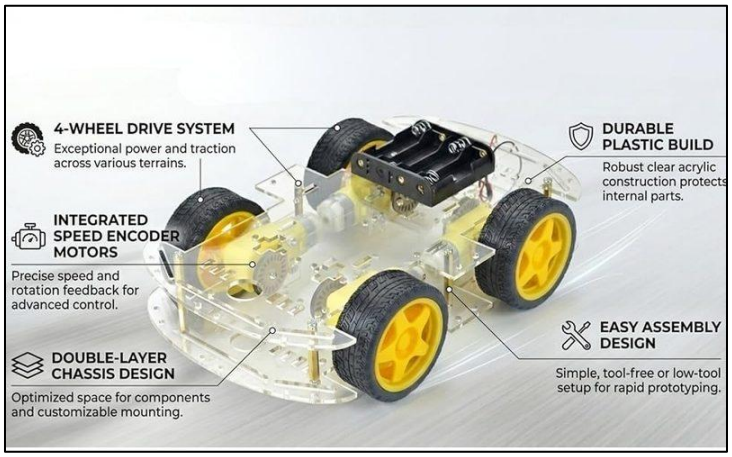


Figure 5.5 — Four-Wheel Acrylic Drive Chassis

A pre-cut transparent acrylic chassis was used as the mechanical platform. It is a double-layer design with mounting cutouts for four TT motors, a central battery compartment, and ample space for the breadboard, microcontrollers, and motor drivers on the upper deck.

Figure 5.6 — 18650 Lithium-Ion Cells

Three rechargeable 18650 Li-ion cells, each rated 3.7 V / 2200 mAh, are connected in series to form a 12.6 V (nominal) battery pack. This single pack supplies the motor drivers directly and, via the buck converter, the 5 V logic rail for the STM32 and NodeMCU.

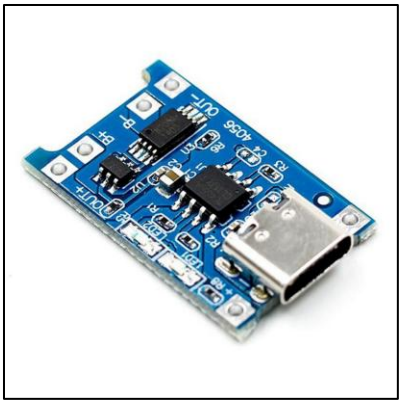


Figure 5.7 — Charging Module

A charging module circuit converts an external power source into a regulated, safe voltage and current to replenish a rechargeable battery. It typically utilizes a Constant-Current/Constant-Voltage (CC/CV) profile to safely charge cells, such as Li-ion, without causing damage.

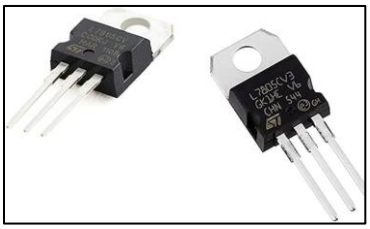


Figure 5.8 — L7805 Linear Voltage Regulator

The L7805 is a TO-220 packaged fixed 5 V linear regulator. It was procured as a backup power-regulation option to the buck converter and is included in the bill of materials for completeness. The final build uses the buck converter for primary regulation owing to its higher efficiency and lower thermal dissipation.

Figure 5.9 — ST-Link V2 SWD Programmer

The ST-Link V2 USB programmer connects to the SWD header of the STM32 Blue Pill and is used to flash and debug the firmware. Four wires (SWDIO, SWCLK, 3.3 V, GND) are connected during programming and disconnected during normal operation.

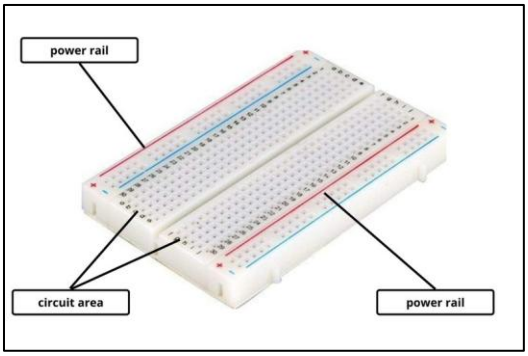


Figure 5.10 — Solderless Breadboard

A standard 400-point solderless breadboard is used to host the STM32, the NodeMCU, and the inter-module wiring during prototyping. The two long power rails on the sides distribute the 5 V and ground signals across all components on the board.

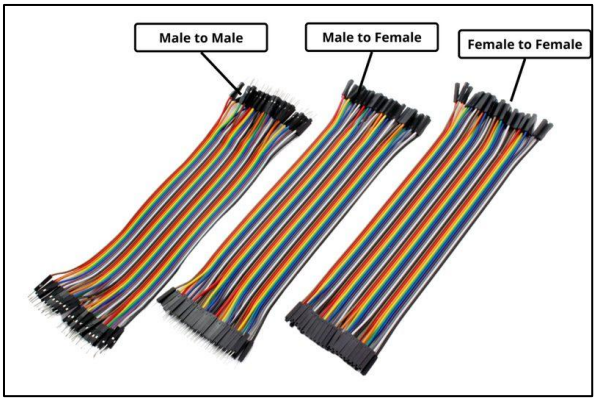


Figure 5.11 — Jumper Wires (M-M, M-F, F-F)

Approximately thirty assorted jumper wires of three types — male-to-male, male-to-female, and female-to-female — are used to make all signal and power interconnections between the boards, drivers, motors, and battery pack. The variety of terminations allows direct connection to header pins, breadboard rows, and screw-terminal blocks.

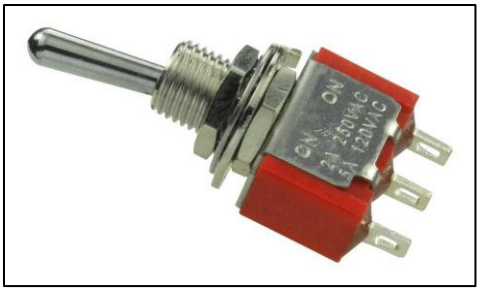


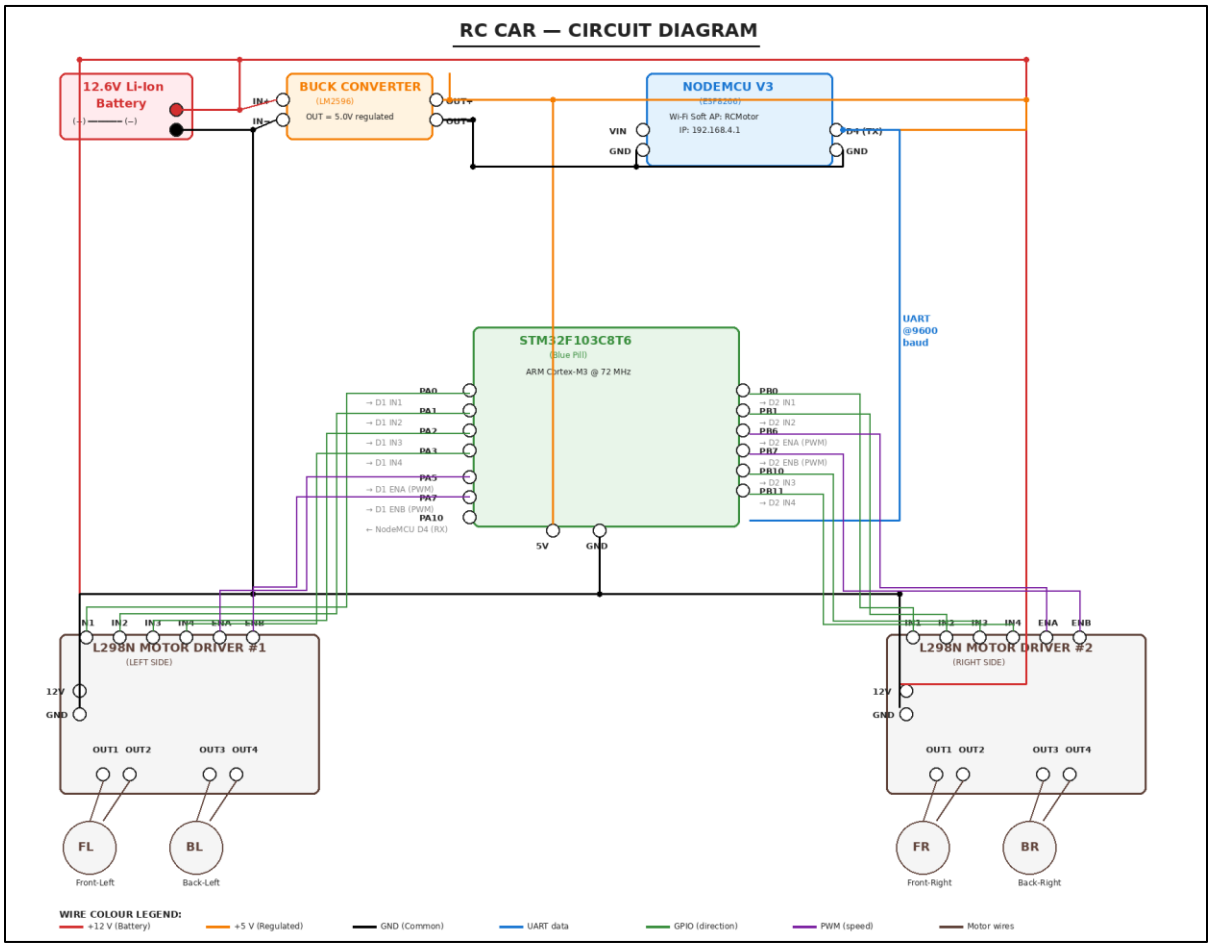
Figure 5.12 — SPDT Toggle Switch (Main Power)

A small SPDT toggle switch was installed inline with the battery positive lead to act as the master power on/off switch. It allows the vehicle to be turned off without disconnecting the battery, preserving the cell life and the wiring connections between sessions.

6. CIRCUIT DESIGN AND CONNECTIONS

This section details the complete pin-level interconnections between the STM32 Blue Pill, the two L298N motor drivers, the NodeMCU, and the power distribution network. The full schematic is shown in Figure 6.1 below, followed by detailed tables for each interface.

Figure 6.1 — Complete Circuit Diagram



6.1 STM32 to Left Motor Driver (L298N #1)

STM32 Pin	L298N #1 Pin (LEFT side)
PA0	IN1 (Front-Left direction)
PA1	IN2 (Front-Left direction)
PA2	IN3 (Back-Left direction)
PA3	IN4 (Back-Left direction)
PA5 (TIM2_CH1)	ENA (Front-Left PWM)
PA7 (TIM3_CH2)	ENB (Back-Left PWM)
GND	GND (common)

6.2 STM32 to Right Motor Driver (L298N #2)

STM32 Pin	L298N #2 Pin (RIGHT side)
PB0	IN1 (Front-Right direction)
PB1	IN2 (Front-Right direction)
PB10	IN3 (Back-Right direction)
PB11	IN4 (Back-Right direction)
PB6 (TIM4_CH1)	ENA (Front-Right PWM)
PB7 (TIM4_CH2)	ENB (Back-Right PWM)
GND	GND (common)

6.3 NodeMCU to STM32 (UART Link)

NodeMCU Pin	STM32 Pin
D4 (GPIO2 / Serial1 TX)	PA10 (USART1 RX)
GND	GND (common)

6.4 Power Distribution

The 12.6 V Li-ion battery feeds both L298N modules directly. A buck converter (LM2596) steps down the same battery voltage to a regulated 5 V supply, which then powers the STM32 (via the 5 V pin) and the NodeMCU (via the VIN pin). All grounds are tied to a common ground rail to ensure consistent signal reference levels across the system.

Source	Destination
Battery + (12.6 V)	L298N #1 12V, L298N #2 12V, Buck IN+
Battery − (GND)	All GND rails
Buck OUT+ (5 V)	STM32 5V pin, NodeMCU VIN pin
Buck OUT− (GND)	Common GND rail

6.5 L298N Jumper Configuration

- 5 V Enable jumper: ON (uses on-board regulator output for logic VCC)
- ENA jumper: REMOVED on both drivers (PWM control via STM32)
- ENB jumper: REMOVED on both drivers (PWM control via STM32)

6.6 Photographs of the Assembled RC Car

Photographs of the fully assembled and wired RC car are presented below. These images show the actual physical implementation of the schematic given in Figure 6.1, with all components mounted on the acrylic chassis, the battery pack secured to the upper deck, and the complete wiring harness routed between the NodeMCU, STM32 (on the breadboard), and the two L298N motor drivers.

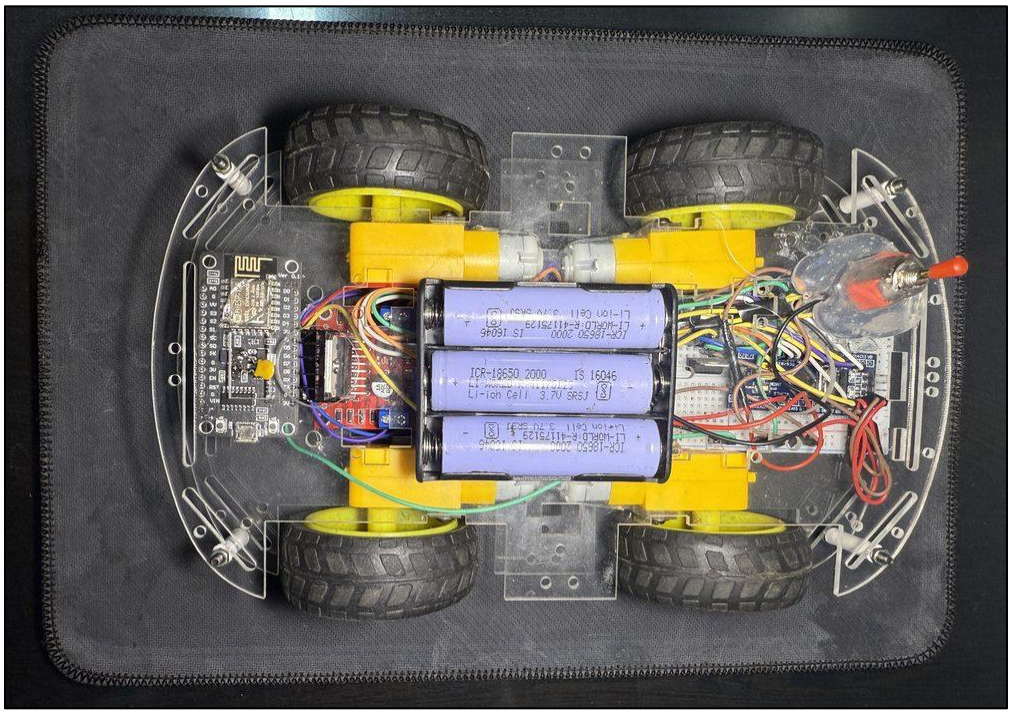


Figure 6.2 — Top view of the assembled RC car

The top view clearly shows the layout of the system. On the left, the NodeMCU is visible alongside one L298N motor driver and the buck converter. The 18650 battery pack sits in the centre between the wheels, and the STM32 on its breadboard occupies the right half of the upper deck. The red power toggle switch is mounted near the front-right corner of the chassis.

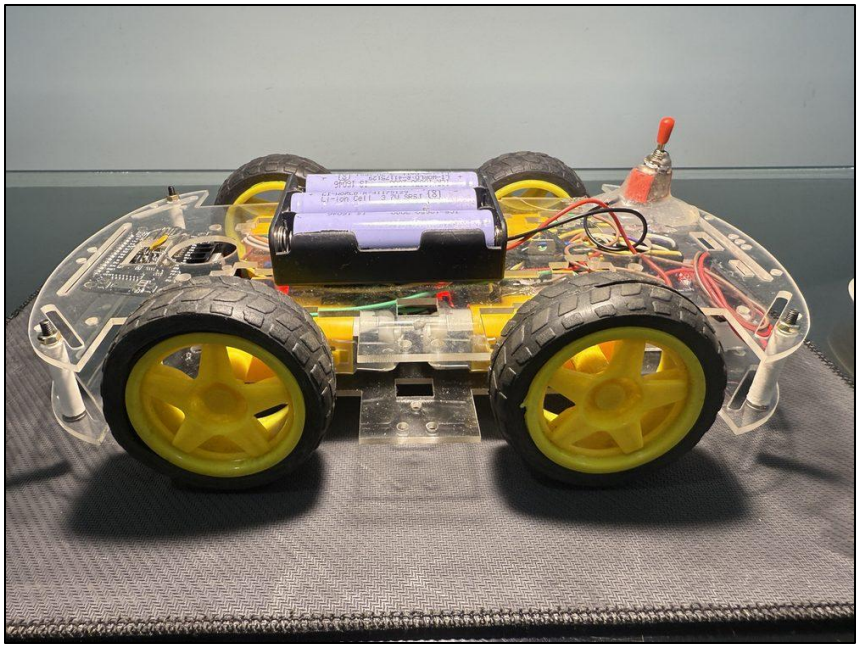


Figure 6.3 — Side view of the assembled RC car

The side view reveals the double-layer chassis arrangement: the four TT motors are mounted to the lower deck between the wheels, while the electronics and the battery sit on the upper deck. The clearance between the two decks accommodates the motor wiring without obstruction.

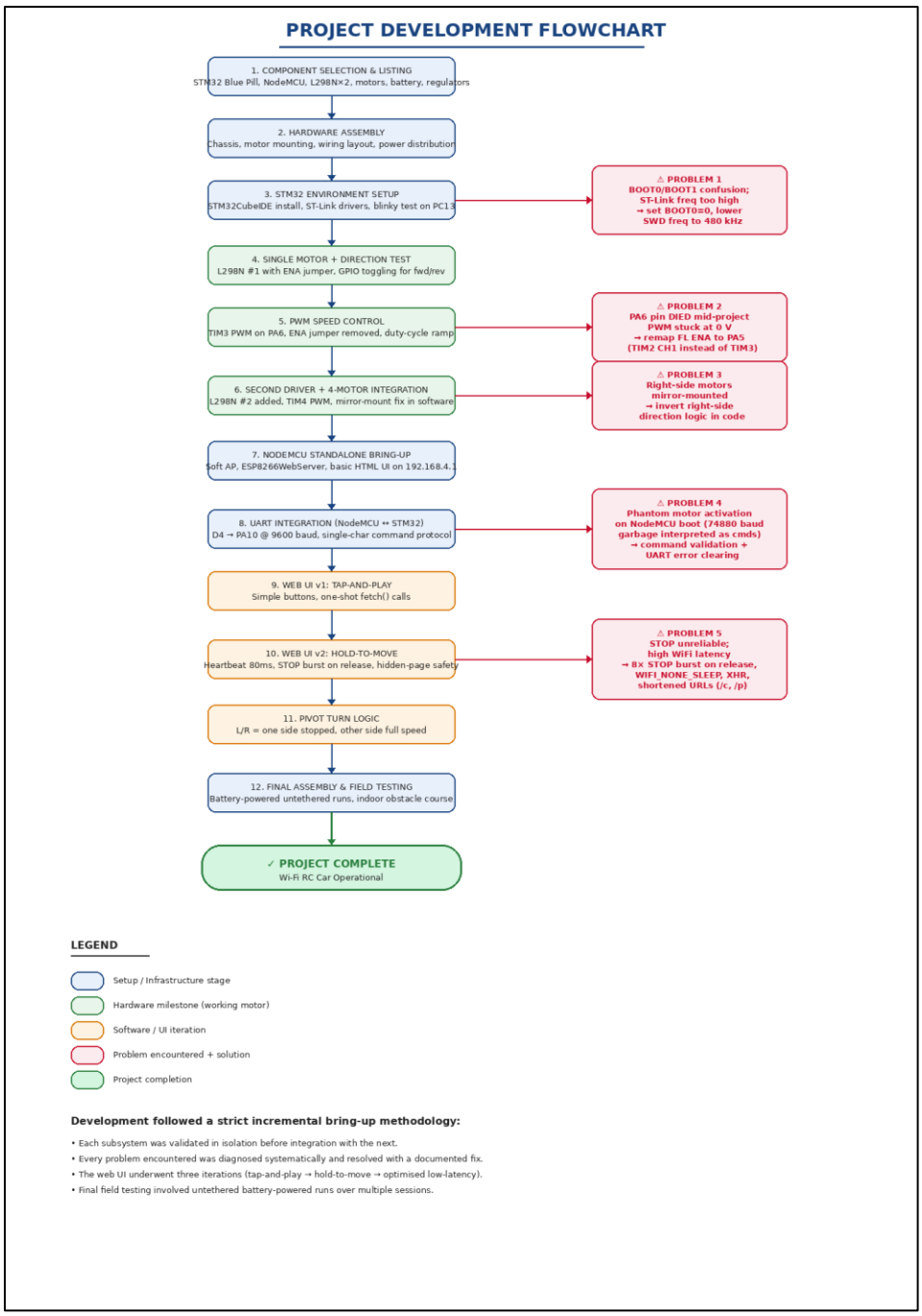
Figure 6.4 — Front view of the assembled RC car

The front view shows the four-wheel-drive arrangement and the centred battery pack mounted directly above the motor assembly. The visible PCB in the foreground is one of the L298N motor drivers, with its prominent black heat-sink and screw-terminal outputs facing forward.

7. PROJECT DEVELOPMENT FLOWCHART

The project was executed using a strict incremental bring-up methodology in which each subsystem was developed, tested, and validated in isolation before being integrated with the next. This approach minimised the search space when debugging issues — if a newly introduced behaviour broke, the most recent integration was always the prime suspect. Figure 7.1 below summarises the twelve major development stages, the five principal problems encountered during the work, and the project's terminal success state.

Figure 7.1 — Project Development Flowchart



8. SOFTWARE IMPLEMENTATION

The software stack consists of two distinct firmware programs running on the two microcontrollers. The STM32 firmware is written in bare-metal C with direct register-level peripheral configuration, while the NodeMCU firmware is written in C++ using the Arduino framework and the ESP8266WebServer library. The two programs communicate over a half-duplex UART link at 9600 baud.

8.1 Development Environment

- STM32CubeIDE 1.x for STM32 firmware development and compilation
- STM32 ST-Link Utility for flashing the compiled binary
- Arduino IDE 2.x with the ESP8266 board package for NodeMCU programming
- ST-Link V2 SWD programmer for STM32 firmware flashing
- USB-TTL CH340G converter for NodeMCU programming and serial debugging

8.2 Communication Protocol

The NodeMCU and STM32 communicate over a half-duplex UART link configured at 9600 baud, 8 data bits, 1 stop bit, no parity. Each command is encoded as a single ASCII character to minimise on-the-wire latency. The supported command set is:

Command	Action
F	Move forward at the current speed
B	Move backward at the current speed
L	Pivot left (left wheels stopped, right wheels forward)
R	Pivot right (right wheels stopped, left wheels forward)
S	Active brake (instant electrical stop)
C	Coast stop (cut power, gradual stop)
1	Set current speed to slow (PWM ≈ 50%)
2	Set current speed to medium (PWM ≈ 80%)
3	Set current speed to fast (PWM 100%)

9. STM32 FIRMWARE SOURCE CODE

The STM32 firmware is written entirely in bare-metal C using direct memory-mapped register access. No HAL (Hardware Abstraction Layer) or LL (Low Layer) libraries are used, which keeps the codebase compact, educationally valuable, and free of unnecessary abstraction layers. The complete source listing follows.

```
#include <stdint.h>

#define RCC_BASE    0x40021000
#define GPIOA_BASE  0x40010800
#define GPIOB_BASE  0x40010C00
#define GPIOC_BASE  0x40011000
#define USART1_BASE 0x40013800
#define TIM2_BASE   0x40000000
#define TIM3_BASE   0x40000400
#define TIM4_BASE   0x40000800

#define RCC_APB2ENR (*(volatile uint32_t*)(RCC_BASE + 0x18))
#define RCC_APB1ENR (*(volatile uint32_t*)(RCC_BASE + 0x1C))

#define GPIOA_CRL   (*(volatile uint32_t*)(GPIOA_BASE + 0x00))
#define GPIOA_CRH   (*(volatile uint32_t*)(GPIOA_BASE + 0x04))
#define GPIOA_ODR   (*(volatile uint32_t*)(GPIOA_BASE + 0x0C))

#define GPIOB_CRL   (*(volatile uint32_t*)(GPIOB_BASE + 0x00))
#define GPIOB_CRH   (*(volatile uint32_t*)(GPIOB_BASE + 0x04))
#define GPIOB_ODR   (*(volatile uint32_t*)(GPIOB_BASE + 0x0C))

#define GPIOC_CRH   (*(volatile uint32_t*)(GPIOC_BASE + 0x04))
#define GPIOC_ODR   (*(volatile uint32_t*)(GPIOC_BASE + 0x0C))

#define USART1_SR   (*(volatile uint32_t*)(USART1_BASE + 0x00))
#define USART1_DR   (*(volatile uint32_t*)(USART1_BASE + 0x04))
#define USART1_BRR  (*(volatile uint32_t*)(USART1_BASE + 0x08))
#define USART1_CR1  (*(volatile uint32_t*)(USART1_BASE + 0x0C))

#define TIM2_CR1    (*(volatile uint32_t*)(TIM2_BASE + 0x00))
#define TIM2_CCMR1  (*(volatile uint32_t*)(TIM2_BASE + 0x18))
#define TIM2_CCER   (*(volatile uint32_t*)(TIM2_BASE + 0x20))
#define TIM2_PSC    (*(volatile uint32_t*)(TIM2_BASE + 0x28))
#define TIM2_ARR    (*(volatile uint32_t*)(TIM2_BASE + 0x2C))
#define TIM2_CCR1   (*(volatile uint32_t*)(TIM2_BASE + 0x34))

#define TIM3_CR1    (*(volatile uint32_t*)(TIM3_BASE + 0x00))
#define TIM3_CCMR1  (*(volatile uint32_t*)(TIM3_BASE + 0x18))
#define TIM3_CCER   (*(volatile uint32_t*)(TIM3_BASE + 0x20))
#define TIM3_PSC    (*(volatile uint32_t*)(TIM3_BASE + 0x28))
#define TIM3_ARR    (*(volatile uint32_t*)(TIM3_BASE + 0x2C))
#define TIM3_CCR2   (*(volatile uint32_t*)(TIM3_BASE + 0x38))

#define TIM4_CR1    (*(volatile uint32_t*)(TIM4_BASE + 0x00))
#define TIM4_CCMR1  (*(volatile uint32_t*)(TIM4_BASE + 0x18))
#define TIM4_CCER   (*(volatile uint32_t*)(TIM4_BASE + 0x20))
#define TIM4_PSC    (*(volatile uint32_t*)(TIM4_BASE + 0x28))
#define TIM4_ARR    (*(volatile uint32_t*)(TIM4_BASE + 0x2C))
#define TIM4_CCR1   (*(volatile uint32_t*)(TIM4_BASE + 0x34))
#define TIM4_CCR2   (*(volatile uint32_t*)(TIM4_BASE + 0x38))

#define FL_IN1 (1<<0)
#define FL_IN2 (1<<1)
#define BL_IN3 (1<<2)
#define BL_IN4 (1<<3)

#define FR_IN1 (1<<0)
#define FR_IN2 (1<<1)
#define BR_IN3 (1<<10)
#define BR_IN4 (1<<11)

#define FULL_SPEED 999
```

```
#define MED_SPEED  800
#define SLOW_SPEED 500

int current_speed = FULL_SPEED;

void delay(volatile uint32_t count) {
    while (count--);
}

void gpio_init() {
    RCC_APB2ENR |= (1<<2) | (1<<3) | (1<<4) | (1<<0) | (1<<14);
    RCC_APB1ENR |= (1<<0) | (1<<1) | (1<<2);

    GPIOA_CRL = 0;
    GPIOA_CRL |= (0x2 << 0);   // PA0 output
    GPIOA_CRL |= (0x2 << 4);   // PA1 output
    GPIOA_CRL |= (0x2 << 8);   // PA2 output
    GPIOA_CRL |= (0x2 << 12);  // PA3 output
    GPIOA_CRL |= (0xB << 20);  // PA5 AF push-pull 50MHz
    GPIOA_CRL |= (0xB << 28);  // PA7 AF push-pull 50MHz

    GPIOA_CRH &= ~(0x00000FF0);
    GPIOA_CRH |=  0x000004B0;  // PA10 input with pull-up
    GPIOA_ODR |=  (1<<10);

    GPIOB_CRL = 0;
    GPIOB_CRL |= (0x2 << 0);   // PB0
    GPIOB_CRL |= (0x2 << 4);   // PB1
    GPIOB_CRL |= (0xB << 24);  // PB6 AF
    GPIOB_CRL |= (0xB << 28);  // PB7 AF

    GPIOB_CRH &= ~(0x0000FF00);
    GPIOB_CRH |= (0x2 << 8);   // PB10
    GPIOB_CRH |= (0x2 << 12);  // PB11

    GPIOC_CRH &= ~(0xF << 20);
    GPIOC_CRH |=  (0x2 << 20); // PC13 LED
}

void usart1_init() {
    USART1_BRR = 833;          // 9600 baud at 8 MHz
    USART1_CR1 = (1<<13) | (1<<3) | (1<<2);
}

void tim2_pwm_init() {
    TIM2_PSC   = 79;
    TIM2_ARR   = 999;
    TIM2_CCMR1 = (0x6 << 4) | (1 << 3);
    TIM2_CCER  = (1 << 0);
    TIM2_CCR1  = 0;
    TIM2_CR1  |= (1 << 0);
}

void tim3_pwm_init() {
    TIM3_PSC   = 79;
    TIM3_ARR   = 999;
    TIM3_CCMR1 = (0x6 << 12) | (1 << 11);
    TIM3_CCER  = (1 << 4);
    TIM3_CCR2  = 0;
    TIM3_CR1  |= (1 << 0);
}

void tim4_pwm_init() {
```

```
    TIM4_PSC   = 79;
    TIM4_ARR   = 999;
    TIM4_CCMR1 = (0x6 << 4) | (1 << 3) | (0x6 << 12) | (1 << 11);
    TIM4_CCER  = (1 << 0) | (1 << 4);
    TIM4_CCR1  = 0;
    TIM4_CCR2  = 0;
    TIM4_CR1  |= (1 << 0);
}

void move_forward(int left_speed, int right_speed) {
    GPIOA_ODR |=  FL_IN1;  GPIOA_ODR &= ~FL_IN2;
    GPIOA_ODR |=  BL_IN3;  GPIOA_ODR &= ~BL_IN4;
    GPIOB_ODR &= ~FR_IN1;  GPIOB_ODR |=  FR_IN2;
    GPIOB_ODR &= ~BR_IN3;  GPIOB_ODR |=  BR_IN4;
    TIM2_CCR1 = left_speed;   TIM3_CCR2 = left_speed;
    TIM4_CCR1 = right_speed;  TIM4_CCR2 = right_speed;
    GPIOC_ODR &= ~(1<<13);
}

void move_backward(int left_speed, int right_speed) {
    GPIOA_ODR &= ~FL_IN1;  GPIOA_ODR |=  FL_IN2;
    GPIOA_ODR &= ~BL_IN3;  GPIOA_ODR |=  BL_IN4;
    GPIOB_ODR |=  FR_IN1;  GPIOB_ODR &= ~FR_IN2;
    GPIOB_ODR |=  BR_IN3;  GPIOB_ODR &= ~BR_IN4;
    TIM2_CCR1 = left_speed;   TIM3_CCR2 = left_speed;
    TIM4_CCR1 = right_speed;  TIM4_CCR2 = right_speed;
    GPIOC_ODR &= ~(1<<13);
}

void pivot_left() {
    GPIOA_ODR &= ~FL_IN1; GPIOA_ODR &= ~FL_IN2;
    GPIOA_ODR &= ~BL_IN3; GPIOA_ODR &= ~BL_IN4;
    TIM2_CCR1 = 0; TIM3_CCR2 = 0;
    GPIOB_ODR &= ~FR_IN1; GPIOB_ODR |=  FR_IN2;
    GPIOB_ODR &= ~BR_IN3; GPIOB_ODR |=  BR_IN4;
    TIM4_CCR1 = FULL_SPEED; TIM4_CCR2 = FULL_SPEED;
    GPIOC_ODR &= ~(1<<13);
}

void pivot_right() {
    GPIOA_ODR |=  FL_IN1; GPIOA_ODR &= ~FL_IN2;
    GPIOA_ODR |=  BL_IN3; GPIOA_ODR &= ~BL_IN4;
    TIM2_CCR1 = FULL_SPEED; TIM3_CCR2 = FULL_SPEED;
    GPIOB_ODR &= ~FR_IN1; GPIOB_ODR &= ~FR_IN2;
    GPIOB_ODR &= ~BR_IN3; GPIOB_ODR &= ~BR_IN4;
    TIM4_CCR1 = 0; TIM4_CCR2 = 0;
    GPIOC_ODR &= ~(1<<13);
}

void stop_brake() {
    TIM2_CCR1 = 0; TIM3_CCR2 = 0;
    TIM4_CCR1 = 0; TIM4_CCR2 = 0;
    GPIOA_ODR |= FL_IN1 | FL_IN2 | BL_IN3 | BL_IN4;
    GPIOB_ODR |= FR_IN1 | FR_IN2 | BR_IN3 | BR_IN4;
    GPIOC_ODR |= (1<<13);
}

void stop_coast() {
    GPIOA_ODR &= ~(FL_IN1 | FL_IN2 | BL_IN3 | BL_IN4);
    GPIOB_ODR &= ~(FR_IN1 | FR_IN2 | BR_IN3 | BR_IN4);
    TIM2_CCR1 = 0; TIM3_CCR2 = 0;
    TIM4_CCR1 = 0; TIM4_CCR2 = 0;
    GPIOC_ODR |= (1<<13);
}

int  usart1_data_available() { return (USART1_SR & (1<<5)); }
```

```
char usart1_read_char()      { return (char)USART1_DR; }
void usart1_clear_errors() {
    volatile uint32_t t;
    t = USART1_SR;
    t = USART1_DR;
    (void)t;
}

int is_valid_command(char c) {
    return (c=='F' || c=='B' || c=='L' || c=='R' ||
            c=='S' || c=='C' ||
            c=='1' || c=='2' || c=='3');
}

int main(void) {
    gpio_init();
    usart1_init();
    tim2_pwm_init();
    tim3_pwm_init();
    tim4_pwm_init();

    stop_coast();
    delay(500000);
    usart1_clear_errors();
    while (usart1_data_available()) usart1_read_char();

    char cmd;
    while (1) {
        while (!usart1_data_available()) { /* idle */ }
        if (USART1_SR & ((1<<3)|(1<<2)|(1<<1)|(1<<0))) {
            usart1_clear_errors();
            continue;
        }
        cmd = usart1_read_char();
        if (!is_valid_command(cmd)) continue;

        switch (cmd) {
            case 'F': move_forward(current_speed, current_speed); break;
            case 'B': move_backward(current_speed, current_speed); break;
            case 'L': pivot_left(); break;
            case 'R': pivot_right(); break;
            case 'S': stop_brake(); break;
            case 'C': stop_coast(); break;
            case '1': current_speed = SLOW_SPEED;
                      move_forward(current_speed, current_speed); break;
            case '2': current_speed = MED_SPEED;
                      move_forward(current_speed, current_speed); break;
            case '3': current_speed = FULL_SPEED;
                      move_forward(current_speed, current_speed); break;
        }
    }
}
```

10. NODEMCU SOURCE CODE

The NodeMCU firmware is written in C++ using the Arduino framework. It performs two main tasks: hosting a Wi-Fi Soft Access Point with SSID "RCMotor", and serving an HTML/CSS/JavaScript-based control panel accessible at http://192.168.4.1 through any modern smartphone browser. The full source listing follows. The HTML page itself is embedded as a raw string literal inside the C++ source to keep the deployment to a single file.

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>

const char* ssid     = "RCMotor";
const char* password = "12345678";

ESP8266WebServer server(80);

const char* htmlPage = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <title>RC Motor Control</title>
  <meta name="viewport" content="width=device-width, initial-scale=1, user-scalable=no, maximum-scale=1">
  <style>
    * { box-sizing: border-box; -webkit-tap-highlight-color: transparent; }
    body {
      font-family: Arial; text-align: center;
      background: #1a1a1a; color: white;
      user-select: none; -webkit-user-select: none;
      touch-action: none; margin: 0; padding: 10px; overflow: hidden;
    }
    h1 { color: #00ff88; font-size: 22px; margin: 10px 0; }
    .btn {
      display: inline-block; padding: 30px 40px; margin: 8px;
      font-size: 28px; font-weight: bold; border: none;
      border-radius: 15px; width: 140px; color: white;
      box-shadow: 0 4px 10px rgba(0,0,0,0.3); touch-action: none;
    }
    .btn.pressed { transform: scale(0.95); filter: brightness(1.4); }
    .fwd { background: #00cc44; } .bwd { background: #cc2200; }
    .lft { background: #0066cc; } .rgt { background: #0066cc; }
    .stp { background: #cc0000; padding: 20px 40px; font-size: 22px; width: 220px; }
    .spd { background: #cc8800; padding: 15px 30px; font-size: 18px; width: 100px; }
    .row { margin: 5px; }
    .status { margin-top: 15px; padding: 10px; background: #333;
              border-radius: 10px; font-size: 13px; }
    .connected { color: #00ff88; } .disconnected { color: #ff4444; }
  </style>
</head>
<body>
  <h1>RC Motor Control</h1>
  <div class="row"><button class="btn fwd" id="btnF">FWD</button></div>
  <div class="row">
    <button class="btn lft" id="btnL">LEFT</button>
    <button class="btn rgt" id="btnR">RIGHT</button>
  </div>
  <div class="row"><button class="btn bwd" id="btnB">BWD</button></div>
  <div class="row" style="margin-top:15px">
    <button class="btn stp" id="btnStop">EMERGENCY STOP</button>
  </div>
  <div class="row" style="margin-top:15px">
    <button class="btn spd" id="btn1">SLOW</button>
    <button class="btn spd" id="btn2">MED</button>
    <button class="btn spd" id="btn3">FAST</button>
  </div>
  <div class="status">
    <span id="status" class="connected">o</span>
    <span id="current">STOPPED</span>
  </div>
  <script>
    let activeCmd = null;
    let heartbeatId = null;

    function send(cmd) {
      const xhr = new XMLHttpRequest();
      xhr.open('GET', '/c?v=' + cmd, true);
```

```
  xhr.timeout = 200;
  xhr.send();
}

window.addEventListener('load', () => { send('S'); });

function startHold(cmd) {
  if (activeCmd === cmd) return;
  activeCmd = cmd;
  send(cmd);
  setTimeout(() => { if (activeCmd === cmd) send(cmd); }, 10);
  setTimeout(() => { if (activeCmd === cmd) send(cmd); }, 25);
  const dirText = { 'F':'FORWARD','B':'BACKWARD','L':'LEFT','R':'RIGHT' };
  document.getElementById('current').innerText = dirText[cmd] || cmd;
  heartbeatId = setInterval(() => {
    if (activeCmd) send(activeCmd);
  }, 80);
}

function stopHold() {
  if (!activeCmd) return;
  activeCmd = null;
  if (heartbeatId) { clearInterval(heartbeatId); heartbeatId = null; }
  send('S');
  setTimeout(() => send('S'), 15);
  setTimeout(() => send('S'), 30);
  setTimeout(() => send('S'), 50);
  setTimeout(() => send('S'), 75);
  setTimeout(() => send('S'), 100);
  setTimeout(() => send('S'), 150);
  setTimeout(() => send('S'), 200);
  document.getElementById('current').innerText = 'STOPPED';
}

function emergencyStop() {
  activeCmd = null;
  if (heartbeatId) { clearInterval(heartbeatId); heartbeatId = null; }
  for (let i = 0; i < 15; i++) {
    setTimeout(() => send('S'), i * 20);
  }
  document.getElementById('current').innerText = 'EMERGENCY STOPPED';
}

function setSpeed(cmd) {
  send(cmd); send(cmd);
  const speedText = { '1':'SLOW','2':'MED','3':'FAST' };
  document.getElementById('current').innerText = speedText[cmd] + ' SET';
  setTimeout(() => {
    if (!activeCmd) {
      send('S'); send('S');
      document.getElementById('current').innerText = 'STOPPED';
    }
  }, 100);
}

function setupButton(id, cmd) {
  const btn = document.getElementById(id);
  const onStart = (e) => { e.preventDefault();
    btn.classList.add('pressed'); startHold(cmd); };
  const onEnd = (e) => { e.preventDefault();
    btn.classList.remove('pressed'); stopHold(); };
  btn.addEventListener('touchstart', onStart, { passive:false });
  btn.addEventListener('touchend',   onEnd,   { passive:false });
  btn.addEventListener('touchcancel',onEnd,   { passive:false });
  btn.addEventListener('mousedown',  onStart);
  btn.addEventListener('mouseup',    onEnd);
```

```
      btn.addEventListener('mouseleave', onEnd);
    }
    setupButton('btnF','F'); setupButton('btnB','B');
    setupButton('btnL','L'); setupButton('btnR','R');

    document.getElementById('btnStop').addEventListener('click', emergencyStop);
    document.getElementById('btnStop').addEventListener('touchstart', (e) => {
      e.preventDefault(); emergencyStop();
    }, { passive:false });

    document.getElementById('btn1').addEventListener('click', () => setSpeed('1'));
    document.getElementById('btn2').addEventListener('click', () => setSpeed('2'));
    document.getElementById('btn3').addEventListener('click', () => setSpeed('3'));

    document.addEventListener('touchmove', (e) => e.preventDefault(), {passive:false});
    document.addEventListener('gesturestart', (e) => e.preventDefault());
    document.addEventListener('contextmenu',  (e) => e.preventDefault());

    document.addEventListener('visibilitychange', () => {
      if (document.hidden) stopHold();
    });
    window.addEventListener('blur',     stopHold);
    window.addEventListener('pagehide', stopHold);

    setInterval(() => {
      const xhr = new XMLHttpRequest();
      xhr.open('GET', '/p', true);
      xhr.timeout = 500;
      xhr.onload  = () => {
        document.getElementById('status').className = 'connected';
        document.getElementById('status').innerText = 'o';
      };
      xhr.onerror = () => {
        document.getElementById('status').className = 'disconnected';
        document.getElementById('status').innerText = 'x';
      };
      xhr.send();
    }, 2000);
  </script>
</body>
</html>
)rawliteral";

void handleRoot() { server.send(200, "text/html", htmlPage); }
void handleCmd() {
  if (server.hasArg("v")) {
    String cmd = server.arg("v");
    Serial1.print(cmd[0]);
  }
  server.send(200, "text/plain", "");
}
void handlePing() { server.send(200, "text/plain", ""); }

void setup() {
  Serial.begin(115200);
  Serial1.begin(9600);
  WiFi.softAP(ssid, password, 1, 0, 4);
  WiFi.setSleepMode(WIFI_NONE_SLEEP);
  Serial.print("IP: "); Serial.println(WiFi.softAPIP());
  server.on("/",  handleRoot);
  server.on("/c", handleCmd);
  server.on("/p", handlePing);
  server.begin();
  Serial.println("Ready!");
}
```

```
void loop() {
  server.handleClient();
}
```

11. WORKING PRINCIPLE

The end-to-end command flow when the user holds the FORWARD button is as follows:

- **Step 1:** The user touches the FORWARD button on the smartphone. The browser fires a touchstart event.
- **Step 2:** The JavaScript handler issues an immediate XMLHttpRequest GET to http://192.168.4.1/c?v=F.
- **Step 3:** The NodeMCU's web server receives the request, parses the v parameter, and writes the single character 'F' to its Serial1 UART (TX on GPIO2).
- **Step 4:** The UART byte travels at 9600 baud (~1 ms transmission time) to the STM32 PA10 pin.
- **Step 5:** The STM32's USART1 receiver flag (RXNE) is set, the firmware reads the byte, validates it against the supported command set, and invokes move_forward().
- **Step 6:** move_forward() sets the direction pins to produce forward rotation on both sides and writes the current_speed value to the timer capture/compare registers (TIM2_CCR1, TIM3_CCR2, TIM4_CCR1, TIM4_CCR2).
- **Step 7:** The PWM outputs on PA5, PA7, PB6, PB7 toggle at 100 Hz with the configured duty cycle. The L298N modules amplify these signals and drive the DC motors.
- **Step 8:** Every 80 ms while the button remains held, the browser repeats the request, keeping the command active. This heartbeat both maintains the motion and provides a built-in liveness signal.
- **Step 9:** When the user releases the button, the browser sends eight STOP requests in 200 ms. The STM32 executes stop_brake(), shorting the motor terminals through the H-bridge for an instant electrical stop.

12. TESTING AND DEBUGGING

The system was developed and tested using an incremental approach, validating each subsystem in isolation before integration. This methodology is captured in the flowchart in Section 7 and is described in detail below.

12.1 STM32 Bring-up

Initial verification was done by flashing a simple PC13 LED blink program. Successful blinking confirmed the SWD programmer connection, the clock tree configuration, and the basic execution flow. The ST-Link V2 was used in normal mode with a 4 MHz SWD frequency for initial bring-up, later increased to 4 MHz once flashing was stable. Issues encountered with BOOT0/BOOT1 jumper configuration were resolved by ensuring BOOT0 was tied LOW for flash-from-internal-memory mode.

12.2 Single Motor Test

A single L298N module was wired to one DC motor with the ENA jumper installed. The firmware was modified to toggle PA0 and PA1 in alternation, producing forward and reverse rotation in three-second

cycles. This validated the basic motor-driver wiring and confirmed that the 12.6 V battery supply was reaching the motor through OUT1 and OUT2 with adequate current capacity.

12.3 PWM Validation

After confirming basic direction control, the ENA jumper was removed and PWM speed control was tested by varying the timer capture/compare register through a ramp from 0 to 999. The motor speed was observed to increase smoothly with PWM duty cycle, confirming correct alternate-function GPIO configuration and timer operation. PWM frequency was set to 100 Hz with 1000-step resolution by configuring the timer prescaler to 79 and the auto-reload to 999 from the 8 MHz system clock.

12.4 Multi-Motor Integration

The second L298N module was then added, and all four motors were connected. A test sequence cycling through forward, stop, backward, and stop phases was used to verify correct direction on all motors. A mirrored physical mounting caused the right-side motors to initially rotate opposite to the left side; this was corrected in software by inverting the right-side direction logic in move_forward() and move_backward(), avoiding the need to remount the motors mechanically.

12.5 NodeMCU Standalone Test

The NodeMCU was programmed in isolation and tested by connecting a phone to the RCMotor access point and accessing 192.168.4.1. The HTML page loaded correctly and button presses were verified using the Arduino Serial Monitor at 115200 baud, where each command character was printed in real time before being forwarded to the (then-disconnected) Serial1 UART.

12.6 End-to-End Integration

Finally, the NodeMCU D4 pin was connected to the STM32 PA10 pin, and the full system was tested through the phone interface. All commands functioned as expected, including pivot turns, emergency stops, and speed switching. The complete vehicle was then placed on the floor, disconnected from any tethered power, and run from the on-board battery for the final field tests.

13. CHALLENGES FACED AND RESOLUTIONS

During the development of this project, the team encountered and systematically resolved a number of non-trivial technical challenges. Each problem is documented below along with its root-cause analysis and the implemented solution.

13.1 PA6 Pin Failure

Mid-way through development, the PA6 pin on the STM32 ceased to produce a PWM output despite the timer configuration being verified correct on the oscilloscope. Investigation showed that the pin's voltage was stuck at 0 V during operation, and even a simple GPIO toggle on PA6 produced no output. The pin was concluded to be physically damaged, likely from a transient overcurrent event earlier in development. The solution was to remap the Front-Left ENA control from PA6 (TIM3_CH1) to PA5 (TIM2_CH1), which required enabling the TIM2 clock and re-initialising the timer for the new channel. The fix was implemented in software with only a single wire moved on the hardware.

13.2 Phantom Motor Activation on NodeMCU Boot

During power-on of the NodeMCU, the motors would occasionally spin without any user input. Analysis revealed that the ESP8266 transmits boot-up debug information over its UART at 74880 baud, which the STM32 (running its USART1 at 9600 baud) interprets as random garbage bytes. Some of these bytes coincidentally matched valid command characters such as 'F' or 'L'. The fix involved adding strict command-character validation (an is_valid_command() helper that ignores anything outside the supported set), flushing the UART buffer on STM32 startup, and clearing UART error flags before each read.

13.3 Stop Command Reliability

Early versions of the web interface sent only a single STOP command on button release. Due to occasional Wi-Fi packet loss, motors would sometimes continue running for several seconds after the user lifted their finger. The solution was to implement a burst-send strategy where eight STOP commands are issued within 200 ms of release, combined with an 80 ms heartbeat during hold so that the absence of further heartbeat itself becomes an indirect stop signal.

13.4 Wi-Fi Latency

The default ESP8266 Wi-Fi sleep mode introduced approximately 150 ms of wake-up latency between consecutive HTTP requests, making the control feel sluggish and unpredictable. Disabling sleep mode using WiFi.setSleepMode(WIFI_NONE_SLEEP) reduced response latency by roughly an order of magnitude, at the cost of slightly higher power consumption — an acceptable trade-off for a battery-powered RC car. URL paths were also shortened from /cmd to /c, and HTTP responses were made empty rather than "OK" to further trim per-request overhead.

13.5 Mirrored Motor Mounting

The right-side motors were mounted mirror-imaged to the left side due to the chassis geometry, which caused them to rotate in opposite directions for the same input pin states. Rather than re-mounting the motors mechanically or rewiring the OUT1/OUT2 lines (which would have required desoldering), the issue was resolved in software by inverting the right-side direction logic in the forward and backward control routines. This is a clean example of software compensation for a mechanical quirk.

14. RESULTS

The completed RC car system demonstrates the following measurable outcomes:

- Reliable Wi-Fi connection range of approximately 15-20 metres in open indoor environments.
- End-to-end control latency from button press to motor response of less than 100 ms under normal conditions.
- Smooth four-wheel forward and backward motion with synchronised motor rotation.
- Tight pivot turns achieved through differential motor control, with one side completely stationary while the opposite side rotates at full speed.
- Three selectable speed presets (slow/medium/fast) operating at approximately 50%, 80%, and 100% PWM duty cycles respectively.

- Reliable stop behaviour with no observed residual motion after button release in repeated test trials.
- Stable operation for approximately 30-40 minutes on a single charge of the 12.6 V Li-ion battery pack under typical load.
- Successful concurrent operation of all four motors, two motor drivers, the STM32, the NodeMCU, and the smartphone client without interference.

15. APPLICATIONS

Although built as an educational project, the system architecture has direct applicability to a wide range of real-world domains:

- Educational robotics platforms for embedded systems and IoT curricula.
- Hobbyist remote-controlled vehicles where smartphone-based control is preferred over dedicated transmitters.
- Surveillance robots when extended with a wireless camera module.
- Indoor delivery robots for restaurants, offices, or warehouses.
- Search and rescue prototypes for confined-space exploration.
- Agricultural drones and field-survey rovers for soil moisture or crop monitoring applications.
- Disability assistance carts that can be controlled remotely or via voice-converted-to-touch commands.

16. FUTURE IMPROVEMENTS

Several enhancements have been identified for future iterations of this project:

- Integration of an HC-SR04 ultrasonic distance sensor for autonomous obstacle avoidance.
- Addition of an ESP32-CAM module for first-person view (FPV) live video streaming to the phone.
- Implementation of a battery voltage monitor using one of the STM32 ADC channels to display remaining charge on the web UI.
- Replacement of the discrete-button UI with an analog joystick element that allows continuously variable speed and steering.
- Migration of the communication layer from HTTP over TCP to UDP for lower latency and reduced overhead.
- Addition of front and rear LED lights and an audible buzzer controllable from the web UI.
- Voice command integration through the smartphone's microphone using the Web Speech API.
- Path recording and playback so the car can autonomously repeat a previously driven route.
- Migration of the firmware to FreeRTOS to allow concurrent handling of sensors, communication, and motor control as separate tasks.

17. CONCLUSION

This project successfully demonstrates the design, implementation, and verification of a fully functional Wi-Fi controlled four-wheel RC car using an STM32F103C8T6 microcontroller, an ESP8266-based NodeMCU module, two L298N motor drivers, and four DC motors. The system integrates real-time bare-

metal motor control on the STM32 with high-level Wi-Fi communication on the NodeMCU through a simple yet robust UART command protocol.

Through the development process, the team gained hands-on experience with a wide array of embedded systems concepts including ARM Cortex-M3 register-level programming, GPIO and timer peripheral configuration, PWM signal generation, UART communication, power regulation using both linear regulators and switching converters, Wi-Fi networking, embedded web server design, mobile-responsive HTML/JavaScript user interface design, and systematic hardware debugging.

The iterative nature of the project, with its many failed attempts and refinement cycles, served as a valuable practical lesson in real-world engineering workflows. Each setback — whether the failure of a GPIO pin, the unexpected boot-up serial chatter from the NodeMCU, or the latency issues introduced by default Wi-Fi sleep modes — required diagnosis, hypothesis testing, and targeted resolution. These experiences reinforce that embedded systems development is as much a debugging discipline as it is a design one.

The final system meets all the original objectives: it is wirelessly controllable, mechanically functional, electrically safe, and software-responsive. It also forms a strong foundation upon which the suggested future improvements can be built. The project has been a comprehensive learning exercise in interdisciplinary embedded-systems engineering and demonstrates that complex, useful systems can be assembled from inexpensive, widely available components when the engineering discipline is applied rigorously.

18. REFERENCES

[1] STMicroelectronics, "STM32F103xx Reference Manual (RM0008)," Revision 21, 2021.

[2] STMicroelectronics, "STM32F103C8T6 Datasheet," 2015.

[3] Espressif Systems, "ESP8266 Technical Reference Manual," Version 1.6, 2020.

[4] STMicroelectronics, "L298 Dual Full-Bridge Driver Datasheet," 2000.

[5] J. Yiu, "The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors," 3rd ed., Newnes, 2014.

[6] M. Margolis, "Arduino Cookbook," 3rd ed., O'Reilly Media, 2020.

[7] Arduino, "ESP8266WiFi and ESP8266WebServer Library Documentation," GitHub, 2023.

[8] STMicroelectronics, "STM32CubeIDE User Guide," UM2609, 2021.

[9] Texas Instruments, "LM2596 Step-Down Voltage Regulator Datasheet," SNVS124E, 2018.

[10] Mozilla Developer Network, "XMLHttpRequest API Reference," developer.mozilla.org, accessed 2026.